第八讲教学设计：MVC综合实战—构建Web应用骨架 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: MVC综合实战：构建Web应用骨架
- **授课时长**: 20分钟
- **适用对象**: 已掌握MVC理论及所有Java Web基础技术，准备进行综合项目开发的学生

一、教学目标

知识目标

1. 掌握分层开发（Layered Architecture）的思想，理解Controller、Service、DAO各层的作用与关系。
2. 熟悉在一个完整请求-响应周期（Request-Response Cycle）中，数据流在MVC三层架构中的完整传递路径。
3. 掌握在MVC模式下，整合运用Servlet, JSP, JavaBean, JDBC及EL/JSTL等多种技术的方法。

能力目标

1. 能够独立搭建一个符合MVC和分层思想的Web应用项目骨架。
2. 能够从零开始，完成一个简单但完整的功能模块（如图书列表展示）的开发。
3. 具备初步的Web系统设计和开发能力，能将孤立的技术点协同组合，解决实际问题。

素质目标

1. 培养整体的、系统的工程化思维，理解软件开发不仅仅是写代码，更是架构设计和流程管理。
2. **[思政融合]** 结合项目开发的全过程，强调**团队合作**（各层如同不同岗位的同事）、**工匠精神**（编写简洁、规范、可维护的代码）和**终身学习**（技术不断发展，要持续更新知识体系）的重要性，鼓励学生关注国产软件和框架，增强为国家科技自立自强贡献力量的意识，以此呼应G12毕业要求。

二、教学重点与难点

教学重点

- 一个完整的请求-响应周期在MVC架构中的完整流程追踪。
- Controller层、Service层、DAO层如何各司其职并协同工作。
- 将所有孤立的技术点（Servlet, JSP, EL, JDBC等）整合到一个项目中的实践方法。

教学难点

- **思维难点**: 从单一技术点的学习思维，转变为观察多个技术点如何在一个项目中协同工作的全局思维。
- **实践难点**: 面对一个空白项目，如何从零开始组织目录结构、创建不同分层的类，并将它们正确地"连接"起来。

三、教学内容与时间安排

第一部分：问题引入 - 从"蓝图"到"建筑" (3分钟)

内容要点

1. **回顾与设问 (Recalling & Questioning):**
   - "上一节课，我们学习了MVC这个伟大的'设计蓝图'，知道了Controller是前台经理，Model是后厨团队，View是摆盘大师。我们有了清晰的理论指导。"
   - **提出挑战**: "今天，我们将从'建筑设计师'转变为'施工队'。我们手头有所有的建筑材料：Servlet（控制器）、JSP（视图）、JavaBean（模型）、JDBC（数据源）、EL/JSTL（视图工具）。问题是：**如何用这些材料，严格按照MVC蓝图，搭建起一座真正的、能运转的Web应用大厦？**"
2. **案例驱动 (Case-Driven):**
   - "我们将以一个非常经典且重要的功能——'**图书列表展示**'为案例。这个功能虽小，但'五脏俱全'，它将带领我们走过一个Web应用从前到后、再从后到前的完整旅程。"

教学方法

- 承上启下，用"蓝图到建筑"的比喻，自然地从理论过渡到实践。
- 项目驱动法，明确本堂课的实践目标，让学习过程更有方向感。

第二部分：架构设计与代码实现之旅 (14分钟)

内容要点

1. **项目骨架搭建 (2分钟)**
   - 在IDE中演示创建项目，并建立规范的包结构：
     - `com.tyust.controller` (存放Servlet)
     - `com.tyust.service` (存放业务逻辑接口与实现)
     - `com.tyust.dao` (存放数据访问接口与实现)
     - `com.tyust.model` (存放JavaBean实体类)
   - 强调"**未写代码，结构先行**"的工程化思想。
2. **代码实现：自底向上 (10分钟)**
   - 采用"**自底向上**"的编码顺序，因为上层依赖于下层。
   - **准备工作：数据库工具类** `DBUtil.java`
     - 为了简化JDBC操作，我们首先创建一个工具类来管理数据库连接和资源关闭。

```
public class DBUtil {
    // 此处省略具体连接参数
    public static Connection getConnection() { /* ... */ }
    public static void close(Connection conn, Statement stmt, ResultSet rs)
{ /* ... */ }
}
```

   - **第一站：Model层 - 定义"货物"**
     - 创建 `Book.java` 类，包含 `id`，`name`，`author` 等属性和对应的getter/setter。

```java
public class Book {
    private int id;
    private String name;
    private String author;
    // Getters and Setters...
}
```

- **第二站：DAO层 - "仓库管理员"**
  - 创建 `BookDao.java` ，编写 `findAll()` 方法，使用**JDBC**和 `DBUtil` 连接数据库，查询所有图书信息，并封装成 `List<Book>` 返回。这是数据的最底层来源。

```java
public class BookDao {
    public List<Book> findAll() {
        List<Book> list = new ArrayList<>();
        Connection conn = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;
        try {
            conn = DBUtil.getConnection();
            String sql = "SELECT * FROM books";
            pstmt = conn.prepareStatement(sql);
            rs = pstmt.executeQuery();
            while (rs.next()) {
                Book book = new Book(/*... set properties from rs
...*/);
                list.add(book);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            DBUtil.close(conn, pstmt, rs);
        }
        return list;
    }
}
```

- **第三站：Service层 - "业务经理"**
  - 创建 `BookService.java` ，编写 `getAllBooks()` 方法，内部调用 `BookDao.findAll()` 。讲解：Service层是业务逻辑的核心，现在它很简单，但未来可能需要在这里进行数据处理、权限判断等复杂操作。

```java
public class BookService {
    private BookDao bookDao = new BookDao();
    public List<Book> getAllBooks() {
        // 未来可能增加缓存逻辑、业务规则校验等
```

```
        return bookDao.findAll();
    }
}
```

- **第四站：Controller层 - "总调度"**
  - 创建 `BookListServlet.java`。
  - 调用 `BookService` 的 `getAllBooks()` 方法，获取图书列表 `List<Book>`。
  - 将列表存入请求域：`request.setAttribute("bookList", list);`
  - **请求转发**给视图：`request.getRequestDispatcher("/bookList.jsp").forward(request, response);`

```
@WebServlet("/listBooks")
public class BookListServlet extends HttpServlet {
    private BookService bookService = new BookService();
    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
        List<Book> bookList = bookService.getAllBooks();
        request.setAttribute("bookList", bookList);

request.getRequestDispatcher("/jsp/bookList.jsp").forward(request,
response);
    }
}
```

- **第五站：View层 - "门面担当"**
  - 创建 `bookList.jsp`。
  - 使用 **EL/JSTL** (`<c:forEach>`) 遍历从Controller传来的 `${bookList}`。
  - 在HTML表格中，使用 `${book.name}`，`${book.author}` 等表达式优美地展示数据。

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
...
<c:forEach items="${bookList}" var="b">
  <tr>
    <td>${b.id}</td>
    <td>${b.name}</td>
    <td>${b.author}</td>
  </tr>
</c:forEach>
...
```

3. **联调与运行 (2分钟)**

- 部署并启动Tomcat。

- 在浏览器访问 `BookListServlet` 对应的URL（例如 `http://localhost:8080/your_project/listBooks`）。
- **流程追踪**:
  1. Tomcat根据URL `/listBooks` 调用 `BookListServlet` 的 `doGet` 方法。
  2. `doGet` 方法中，`bookService.getAllBooks()` 被调用。
  3. `BookService` 调用 `bookDao.findAll()`。
  4. `BookDao` 通过 `DBUtil` 连接数据库，执行SQL查询，并将结果封装成 `List<Book>` 返回。
  5. 该List被 `setAttribute` 到 `request` 对象中。
  6. 请求被 `forward` 到 `bookList.jsp`。
  7. JSP引擎渲染页面，`<c:forEach>` 标签迭代 `request` 中的 `bookList`，生成HTML表格。
  8. 最终的HTML响应被发送到浏览器。
- 成功展示从数据库中查询出的图书列表页面，宣告我们第一个完整的MVC应用构建成功！

教学方法

- **现场编码 (Live Coding)**：一步步、一层层地编写代码，让学生完整地看到项目从无到有的过程。
- **流程追踪法**：每写完一层，都向学生解释它在整个数据流中的位置和作用，强化全局观。

第三部分：总结与升华 - 我们的征途 (3分钟)

总结要点

1. **回顾旅程**: 用一张最终的、完整的流程图，带领学生重走一遍请求从浏览器发出，到各层流转，最终返回响应的完整路径。
2. **核心思想**: MVC的精髓在于**分层**与**解耦**，它让我们的代码结构清晰，易于维护和扩展。
3. **技术串联**: 本次课将之前所有零散的知识点成功地串联成一个有机的整体，形成了一个知识闭环。

思政升华 (Competition Highlight)

- "同学们，今天我们构建的虽然只是一个小小的图书列表功能，但它背后体现了大型软件工程的思想。**DAO层**像是一线的材料供应商，**Service层**是加工和处理核心部件的技术专家，**Controller层**是项目经理，**View层**是产品设计师。只有大家各司其职、紧密**合作**，项目才能成功。"
- "编写这样结构清晰、命名规范的代码，本身就是一种**工匠精神**的体现。希望大家未来无论是参与开源项目，还是进入企业工作，都能秉持这种精神。"
- "我们今天的课程到此就告一段落了，但这绝不是终点。技术世界日新月异，希望大家能保持**终身学习**的热情，未来用我们掌握的知识，为国家的软件产业和科技自立贡献自己的一份力量！"

四、教学方法与手段

教学方法

1. **项目驱动法 (Project-Driven)**: 以"图书列表"完整项目引导
2. **现场编码法 (Live Coding)**: 自底向上逐层编写代码
3. **分层讲解法 (Layer-by-Layer)**: Model→DAO→Service→Controller→View
4. **流程追踪法 (Process Tracing)**: 追踪完整请求处理流程
5. **双语教学法 (Bilingual Teaching)**:
   - 课堂采用英文授课，关键技术术语提供中英文对照

- 重要概念（Layered Architecture, Data Flow, Service Layer, DAO Pattern）首次出现时给出英文表达和中文释义
- 鼓励学生使用英文术语描述分层架构
- 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 数据库 (Database: MySQL/MariaDB)
- 多媒体课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否跟上编码节奏，理解"自底向上"的开发顺序。
- **终结性评价**: 能否独立画出一个完整MVC应用的请求处理流程图；能否用自己的话解释C、S、D三层各自的作用。

六、课后拓展

实践作业

1. **功能扩展**: 在今天完成的"图书列表展示"项目基础上，增加"**添加图书**"的功能。
   - 创建一个 `addBook.jsp` 页面作为表单（View），包含书名、作者等输入框。
   - 创建一个 `AddBookServlet` 作为控制器（Controller），接收POST请求，获取表单参数，调用Service层进行处理。
   - 在 `BookService` 和 `BookDao` 中增加 `addBook(Book book)` 方法，执行数据库的INSERT操作。
   - 完成添加后，**重定向**到 `BookListServlet`（`/listBooks`），以重新查询并刷新列表页面。
2. **项目挑战**: 尝试独立设计并实现一个"用户登录与列表展示"的完整MVC项目。